

1) Virtual Addressing

```
nikhilreddy@Hp: ~  
GNU nano 8.7.1  
#include <stdio.h>  
  
int main() {  
    int x = 10;  
  
    printf("Value of x = %d\n", x);  
    printf("Virtual address of x = %p\n", (void *)&x);  
  
    return 0;  
}
```

```
nikhilreddy@Hp:~$ gcc h1.c -o h1  
nikhilreddy@Hp:~$ ./h1  
Value of x = 10  
Virtual address of x = 0x7fff86e0e0e4
```

2)Memory Layout

```
nikhilreddy@Hp: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <stdlib.h>  
  
int global = 10;  
int bss_variable;  
  
int main() {  
  
    int local = 20;  
  
    int *heap = malloc(sizeof(int));  
    *heap = 30;  
  
    printf("Code address : %p\n", (void *)main);  
    printf("Data address : %p\n", (void *)&global);  
    printf("BSS address : %p\n", (void *)&bss_variable);  
    printf("Heap address : %p\n", (void *)heap);  
    printf("Stack address : %p\n", (void *)&local);  
  
    free(heap);  
  
    return 0;  
}
```

```
nikhilreddy@Hp:~$ nano address.c
nikhilreddy@Hp:~$ gcc address.c -o address
nikhilreddy@Hp:~$ ./address
Code address : 0x5dff51d151a9
Data address : 0x5dff51d18010
BSS address : 0x5dff51d18018
Heap address : 0x5dff74c76010
Stack address : 0x7ffeb4285d1c
```

3)Demand Paging

```
nikhilreddy@Hp: ~
GNU nano 8.7.1
#include <stdio.h>

int main() {

    int page_table[4] = {1, -1, 3, -1};

    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}

nikhilreddy@Hp:~$ gcc h2.c -o h2
nikhilreddy@Hp:~$ ./h2
Enter page number (0-3): 2
Page is already in memory.
Physical frame = 3
```

4)Address Translation

```
nikhilreddy@Hp: ~  
GNU nano 8.7.1  
#include <stdio.h>  
  
int main() {  
  
    int page_table[] = {3, 5, 1, 7};  
    int page_size = 100;  
  
    int virtual_address = 250;  
  
    int page = virtual_address / page_size;  
    int offset = virtual_address % page_size;  
  
    int frame = page_table[page];  
  
    int physical_address = frame * page_size + offset;  
  
    printf("Virtual Address = %d\n", virtual_address);  
    printf("Page Number      = %d\n", page);  
    printf("Offset           = %d\n", offset);  
    printf("Frame Number      = %d\n", frame);  
    printf("Physical Address = %d\n", physical_address);  
  
    return 0;  
}  
  
nikhilreddy@Hp:~$ gcc h3.c -o h3  
nikhilreddy@Hp:~$ ./h3  
Virtual Address = 250  
Page Number      = 2  
Offset           = 50  
Frame Number      = 1  
Physical Address = 150
```

5)Memory Allocation

```
nikhilreddy@Hp: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <stdlib.h>  
  
int main() {  
    int *p;  
  
    p = malloc(5 * sizeof(int));  
  
    if (p == NULL) {  
        printf("Memory allocation failed\n");  
        return 1;  
    }  
  
    for (int i = 0; i < 5; i++) {  
        p[i] = (i + 1) * 10;  
    }  
  
    for (int i = 0; i < 5; i++) {  
        printf("%d ", p[i]);  
    }  
  
    printf("\n");  
  
    free(p);  
  
    return 0;  
}
```

```
nikhilreddy@Hp:~$ nano h4.c  
nikhilreddy@Hp:~$ gcc h4.c -o h4  
nikhilreddy@Hp:~$ ./h4  
10 20 30 40 50
```

6) Segmentation Fault

```
nikhilreddy@Hp: ~  
GNU nano 8.7.1  
#include <stdio.h>  
  
int main() {  
    int *p = NULL;  
    printf("Before accessing memory\n");  
    *p = 100;  
    printf("After accessing memory\n");  
    return 0;  
}  
  
nikhilreddy@Hp:~$ nano h5.c  
nikhilreddy@Hp:~$ gcc h5.c -o h5  
nikhilreddy@Hp:~$ ./h5  
Before accessing memory  
Segmentation fault ./h5
```